ĐẠI HỌC TÔN ĐỨC THẮNG
TON DUC THANG UNIVERSITY

502094 - Software Deployment, Operations And Maintenance

GitHub Actions: From Beginners to Pros

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Introduction to GitHub Actions

- What **GitHub Actions** is and why it exists
- How it fits into **CI/CD** and modern DevOps
- Why backend/DevOps engineers should care
- Roadmap:
  - Fundamentals → Intermediate → Pro-level features

What Is GitHub Actions?

- **GitHub Actions** is:
  - A built-in automation platform inside GitHub
  - Lets you define **workflows** triggered by Git events
- Typical use cases:
  - Continuous Integration (CI)
  - Continuous Delivery/Deployment (CD)
  - Code quality checks and security scanning
  - Scheduled maintenance and automation tasks

CI/CD in Modern DevOps

- **Continuous Integration (CI)**:
  - Automatically build, test, and analyze code on every change
- **Continuous Delivery/Deployment (CD)**:
  - Automatically package, release, and deploy software
- GitHub Actions:
  - Connects **Git** events (push, PR) to automated pipelines
  - Reduces manual work and human errors

Why GitHub Actions vs Other CI Systems?

- Compared to GitLab CI, Jenkins, CircleCI:
  - Native integration with GitHub repositories
  - No separate server or plugins to manage
  - Huge **Marketplace** of reusable actions
- Problems it solves:
  - Fragmented tooling
  - Manual release steps
  - Inconsistent build environments

Problems GitHub Actions Solves

- Before GitHub Actions:
  - Developers manually run tests on laptops
  - Releases depend on one “build master” person
  - CI/CD is configured in separate systems (Jenkins, etc.)
- With GitHub Actions:
  - Workflows are **version-controlled YAML files**
  - Every push/PR is validated consistently
  - Build and deployment logic is transparent to the whole team

PART I — The Fundamentals

- Focus:
  - Core concepts & terminology
  - YAML basics
  - First workflows
  - Using Marketplace actions
  - Language-specific CI examples

Core Concepts & Terminology

- Key concepts in GitHub Actions:
  - **Workflow**
  - **Job**
  - **Step**
  - **Action**
  - **Runner**
  - **Event**
  - **Artifacts & logs**

Workflow, Jobs, Steps

- **Workflow**:
  - Top-level CI/CD pipeline, defined in YAML
  - Located in `.github/workflows/` directory
- **Job**:
  - A group of steps that run on the same runner
  - Can run in parallel with other jobs
- **Step**:
  - Single task: run a command or use an action
  - Steps within a job run sequentially

Actions and Runners

- **Action**:
  - Reusable unit of work
  - Examples:
    - `actions/checkout@v4`
    - `actions/setup-node@v4`
- **Runner**:
  - Machine that executes jobs
  - Types:
    - **GitHub-hosted**: `ubuntu-latest`, `windows-latest`, `macos-latest`
    - **Self-hosted**: your own VM or server

Events and Triggers

- **Events** trigger workflows:
  - `push`
  - `pull_request`
  - `schedule` (cron)
  - `workflow_dispatch` (manual trigger)
- Example:

```yaml
on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]
```

Artifacts & Logs

- **Logs**:
  - Each job/step produces detailed logs
  - Viewable in GitHub Actions UI
- **Artifacts**:
  - Files produced by workflows:
    - Build outputs (JAR, Docker manifests)
    - Coverage reports
    - Screenshots from tests
  - Can be downloaded or consumed by later jobs

Understanding YAML for GitHub Actions

- GitHub Actions workflows are written in **YAML**
- YAML basics:
  - Indentation-based structure
  - Simple, human-readable
- We must avoid:
  - Tabs
  - Inconsistent indentation

YAML Structure: Lists and Dictionaries

- **Dictionary (map)**:

```yaml
name: Node.js CI
runs-on: ubuntu-latest
```

- **List**:

```yaml
steps:
  - name: First step
    run: echo "Hello"
  - name: Second step
    run: echo "World"
```

- Combination of both builds complex workflow definitions

GitHub Actions Syntax Rules

- Workflows must be placed under:
  - `.github/workflows/ci.yml`
- Top-level keys:
  - `name`, `on`, `jobs`
- All keys are case-sensitive
- Indentation:
  - Typically two spaces per level
  - Never use tabs

Expression Syntax: ${{ }}

- GitHub Expressions:
  - Encapsulated in `${{ ... }}`
- Common uses:
  - Accessing context values:
    - `${{ github.sha }}`
    - `${{ github.ref_name }}`
  - Accessing secrets:
    - `${{ secrets.DOCKER_PASSWORD }}`
  - Conditional logic:
    - `if: ${{ success() && github.ref == 'refs/heads/main' }}`

Context Objects Overview

- Useful contexts:
  - `github.*`:
    - Repository, actor, branch, commit SHA
  - `env.*`:
    - Environment variables defined in workflow
  - `inputs.*`:
    - Parameters passed to reusable workflows
  - `secrets.*`:
    - Encrypted values from GitHub Secrets
  - `steps.*`:
    - Outputs from previous steps in same job

Writing Your First Workflow

- Goal:
  - Run tests on every push and PR
- Minimal workflow:

```yaml
name: Basic CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "CI is running on commit ${{ github.sha }}"
```

Explaining the First Workflow

- `name: Basic CI`:
  - Appears in the Actions tab
- `on: [push, pull_request]`:
  - Triggers on both events
- `jobs.test`:
  - A job named `test`
- `runs-on: ubuntu-latest`:
  - Use a Linux VM managed by GitHub
- Steps:
  - Checkout code
  - Echo the commit SHA

Checking Workflow Logs

- After pushing:
  - Go to **Actions** tab in GitHub
  - Select `Basic CI` workflow
  - Click on a run:
    - See each job and step
    - View logs, environment information
- Debugging:
  - Red (failed) step:
    - Expand to see error messages

Fixing First-Time Errors

- Common errors:
  - Indentation mistakes in YAML
  - Wrong action version (e.g., `@v3` vs `@v4`)
  - Missing `run:` or `uses:` under steps
- Strategy:
  - Check error message in logs
  - Use YAML linter locally or in IDE
  - Compare with working workflow examples

Using Marketplace Actions

- **GitHub Marketplace**:
  - Large catalog of prebuilt actions
- Types of actions:
  - JavaScript actions
  - Docker container actions
  - Composite actions
- Benefits:
  - Reuse community-tested logic
  - Focus on your business logic, not plumbing

Evaluating Marketplace Actions

- Before using an action:
  - Check:
    - Number of stars
    - Recent updates and maintenance
    - Documentation quality
    - Source repository
  - Example safe choices:
    - `actions/checkout@v4`
    - `actions/setup-node@v4`
    - `actions/setup-java@v4`

Avoiding Unsafe Actions

- Risks:
  - Actions can run arbitrary code on your runner
- Best practices:
  - Prefer official `actions/*` for critical tasks
  - Pin to a specific version (e.g., `@v4`) or commit SHA
  - Do not use untrusted actions for secrets-heavy steps
  - Review source code of third-party actions when possible

Common Official Actions

- **actions/checkout@v4**:
  - Clone the repo into the runner
- **actions/setup-node@v4**:
  - Install Node.js with optional caching
- **actions/setup-java@v4**:
  - Install JDK with optional Maven/Gradle cache
- **actions/upload-artifact@v4**:
  - Upload build outputs
- **actions/cache@v4**:
  - Cache dependencies

Running CI for Popular Languages

- GitHub Actions supports:
  - Node.js (npm, pnpm, yarn)
  - Java (Maven, Gradle)
  - Python (venv, poetry)
  - .NET (SDK)
  - Ruby, Go, PHP, and more
- Approach:
  - Use language-specific setup actions
  - Run standard build/test commands

Node.js CI Example (npm)

```yaml
jobs:
  node-ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: npm
      - run: npm ci
      - run: npm test -- --coverage
      - run: npm run lint
```

- Scenario:
  - Backend API written in Node.js/Express

Spring Boot CI Example (Maven)

```yaml
jobs:
  spring-ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: "temurin"
          java-version: "17"
          cache: maven
      - run: ./mvnw -B clean test
      - run: ./mvnw -B package
```

- Result:
  - JAR file created in `target/` directory
  - Ready to upload as artifact or build Docker image

Python CI Example

```yaml
jobs:
  python-ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: |
          python -m venv .venv
          source .venv/bin/activate
          pip install -r requirements.txt
      - name: Run tests
        run: |
          source .venv/bin/activate
          pytest --maxfail=1 --disable-warnings
```

.NET CI Example

```yaml
jobs:
  dotnet-ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Setup .NET
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: "8.0.x"
      - name: Restore
        run: dotnet restore
      - name: Build
        run: dotnet build --configuration Release --no-restore
      - name: Test
        run: dotnet test --no-build --verbosity normal
```

- Applicable to: ASP.NET Core backend services

Multi-Language Monorepo Example

- Monorepo structure:
  - `backend-node/`
  - `backend-java/`
  - `frontend-react/`
- Separate jobs per folder:

```yaml
jobs:
  node-backend: # runs Node.js workflow in backend-node
  java-backend:  # runs Spring workflow in backend-java
  frontend: # runs React build/tests
```

- Benefit:
  - Clear responsibilities and parallel execution

Working with Secrets & Environment Variables

- Secrets and env vars are central to:
  - Connecting to registries
  - Calling APIs
  - Encrypting sensitive configuration
- Need to balance:
  - Convenience and **security**

GitHub Secrets Store

- Add secrets in:
  - **Settings → Secrets and variables → Actions**
- Examples:
  - `DOCKER_USERNAME`
  - `DOCKER_PASSWORD`
  - `AWS_ACCESS_KEY_ID`
  - `AWS_SECRET_ACCESS_KEY`
- Usage in workflows:

```
env:
  DOCKER_PASSWORD: ${{ secrets.DOCKER_PASSWORD }}
```

Masking Secrets in Logs

- GitHub automatically:
  - Masks secret values in logs
- Good practices:
  - Never echo secrets directly
  - Avoid writing secrets to files not needed
- If logs show `***`:
  - It indicates a secret was used and masked

Environment Variables: Levels

- **Workflow-level**:

```yaml
env:
  NODE_ENV: test
```

- **Job-level**:

```yaml
jobs:
  build:
    env:
      LOG_LEVEL: debug
```

- **Step-level**:

```yaml
- name: Print API URL
  run: echo "$API_URL"
  env:
    API_URL: https://api.dev.myshop.vn
```

.env Files (With Caution)

- Some projects use `.env`:
  - `API_URL=https://api.dev.myshop.vn`
- In CI:
  - You can create `.env` files in steps:

```
echo "API_URL=https://api.dev.myshop.vn" >> .env
```

- Security warning:
  - Never commit secret `.env` files to Git
  - Prefer GitHub Secrets for sensitive data

Security Considerations Around Secrets

- Principles:
  - **Least privilege**:
    - Provide minimal permissions needed
  - Rotate secrets regularly
  - Use scoped personal access tokens
- For deployments:
  - Consider **OIDC** with cloud providers (advanced topic)
  - Avoid long-lived static credentials

PART II — Intermediate Topics

- Matrix builds & multi-platform testing
- Dependency caching
- Artifacts and workflow outputs
- Building Docker images
- Code quality & PR automation
- Scheduled workflows

Matrix Builds & Multi-Platform Testing

- Use **matrix strategies** to:
  - Test across multiple OS
  - Test multiple language versions
- Example:

```yaml
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    node-version: [18, 20]
```

Matrix Build Example

```yaml
jobs:
  test-matrix:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest]
        node-version: [18, 20]

    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
      - run: npm ci
      - run: npm test
```

- This tests: Node.js 18 & 20 on both Ubuntu and Windows

Excluding Matrix Combinations

- Sometimes not all combinations make sense:

```yaml
strategy:
  matrix:
    os: [ubuntu-latest, windows-latest]
    node-version: [16, 18, 20]
    exclude:
      - os: windows-latest
        node-version: 16
```

- Avoids: Running tests where you know they are unsupported

Dependency Caching Strategies

- Why cache?
  - Speed up builds by reusing downloaded dependencies
- Common caches:
  - Node.js: `~/.npm` or `node_modules`
  - Maven: `~/.m2/repository`
  - Gradle: `~/.gradle/caches`
  - pip: `~/.cache/pip`

Caching npm Dependencies

```yaml
- name: Cache npm
  uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ hashFiles('package-lock.json') }}
    restore-keys: |
      npm-
```

- Key:
  - Changes when `package-lock.json` changes
- Effect:
  - Faster `npm ci` on subsequent runs

Caching Maven Dependencies

```yaml
- name: Cache Maven repository
  uses: actions/cache@v4
  with:
    path: ~/.m2/repository
    key: maven-${{ hashFiles('**/pom.xml') }}
    restore-keys: |
      maven-
```

- Typically combined with `actions/setup-java@v4`
- Reduces time spent downloading Maven artifacts

Maintaining Reproducible Builds

- Danger:
  - Over-caching can hide dependency changes
- Best practices:
  - Use lockfiles (e.g., `package-lock.json`)
  - Invalidate cache on relevant file changes
  - Occasionally clear caches when debugging build issues

Artifacts & Workflow Outputs

- Artifacts:
  - Files persisted after jobs finish
- Workflow outputs:
  - Values computed in one job and used in another
- Use cases:
  - Build once, test and deploy in separate jobs

Uploading and Downloading Artifacts

```yaml
- name: Upload JAR
  uses: actions/upload-artifact@v4
  with:
    name: spring-boot-app
    path: target/*.jar
```

- In another job:

```yaml
- name: Download JAR
  uses: actions/download-artifact@v4
  with:
    name: spring-boot-app
```

- Useful for separated **build** and **deploy** jobs

Passing Outputs Between Jobs

- In producer job:

```yaml
jobs:
  build:
    outputs:
      image-tag: ${{ steps.meta.outputs.tag }}
    steps:
      - id: meta
        run: echo "tag=${GITHUB_SHA}" >> "$GITHUB_OUTPUT"
```

- In consumer job:

```yaml
jobs:
  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying image tag ${{ needs.build.outputs.image-tag }}"
```

Building Docker Images in GitHub Actions

- Steps:
  - Log in to registry (Docker Hub or GHCR)
  - Build image with `docker build`
  - Tag with:
    - `latest`
    - `${{ github.sha }}`
  - Push to registry

Docker Build Workflow Example

```yaml
- name: Log in to Docker Hub
  uses: docker/login-action@v3
  with:
    username: ${{ secrets.DOCKER_USERNAME }}
    password: ${{ secrets.DOCKER_PASSWORD }}

- name: Build Docker image
  run: docker build -t mvmanh/tdtu-api:${{ github.sha }} .

- name: Tag latest
  run: docker tag mvmanh/tdtu-api:${{ github.sha }} mvmanh/tdtu-api:latest

- name: Push images
  run: |
    docker push mvmanh/tdtu-api:${{ github.sha }}
    docker push mvmanh/tdtu-api:latest
```

Docker Buildx and Multi-Architecture

- For multi-arch images (e.g., amd64 + arm64):

```yaml
- uses: docker/setup-buildx-action@v3
- uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: mvmanh/tdtu-api:${{ github.sha }}
    platforms: linux/amd64,linux/arm64
```

- Use case:
  - Support both x86 servers and ARM devices (e.g., Raspberry Pi)

Code Quality & Security Scanning

- Integrate:
  - ESLint, Prettier, TypeScript
  - Checkstyle, SpotBugs
  - Dependabot updates
  - GitHub CodeQL scanning
- Purpose:
  - Enforce **quality gates** before merging

PR Automation & Branch Protection

- Combine:
  - GitHub Actions workflows
  - GitHub branch protection rules
- Capabilities:
  - Require CI checks to pass
  - Require at least one review
  - Prevent direct pushes to `main`

Auto-Labeling and Reviewer Assignment

- Example action:
  - `actions/labeler` for auto-labeling PRs
- Example config:

```yaml
- uses: actions/labeler@v5
  with:
    repo-token: ${{ secrets.GITHUB_TOKEN }}
```

- Auto-assign reviewers:
  - Use community actions that read `CODEOWNERS`
  - Or custom automation via GitHub Apps

Scheduled Workflows

- Use `schedule` event with cron syntax:

```yaml
on:
  schedule:
    - cron: "0 18 * * *"
```

- Example use cases:
  - Nightly test runs
  - Security scans
  - Automatic dependency checks
  - Cleaning up old preview environments

PART III — Advanced Topics (Pro Level)

- Self-hosted runners
- Reusable workflows
- Composite actions
- Advanced pipeline architectures
- Security and optimization

Self-Hosted Runners

- Why use self-hosted?
  - Need access to internal resources (private VPC, on-prem DB)
  - Custom hardware or GPU
  - Avoid GitHub-hosted runner limits
- Trade-offs:
  - You must manage:
    - OS updates
    - Security patches
    - Scaling

Installing a Self-Hosted Runner (Linux)

- Create runner in **Settings → Actions → Runners**
- On Linux server:

```
mkdir actions-runner && \
  cd actions-runner

curl -o actions-runner.tar.gz -L \
  https://github.com/actions/runner/.../actions-runner-linux-x64-2.322.0.tar.gz

tar xzf actions-runner.tar.gz

./config.sh \
  --url https://github.com/mvmanh/tdtu-devops \
  --token YOUR_RUNNER_TOKEN

./run.sh
```

- The repo can now target: `runs-on: self-hosted`

Auto-Scaling Self-Hosted Runners

- Patterns:
  - Use a pool of runners on:
    - Kubernetes
    - Auto-Scaling Groups in AWS
  - Runners are created per job and destroyed after
- Benefits:
  - Isolation per workflow
  - Better security
  - Cost-efficient scaling

Reusable Workflows (DRY Principle)

- Problem:
  - Many repos share similar CI logic
- Solution:
  - Define **reusable workflows** in a central repo
- Use `workflow_call`:

```yaml
on:
  workflow_call:
    inputs:
      node-version:
        required: true
        type: string
```

Calling a Reusable Workflow

- In consumer repo:

```yaml
jobs:
  node-ci:
    uses: org/shared-ci/.github/workflows/node-ci.yml@v1
    with:
      node-version: "20"
```

- Benefits:
  - One place to maintain CI standards
  - Easy to roll out improvements across repos

Composite Actions: Create Your Own Action

- **Composite action**:
  - A reusable action composed of multiple steps
- Structure:
  - Repo with `action.yml`:

```yaml
name: "Setup Node with Cache"
runs:
  using: "composite"
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
        cache: npm
    - run: npm ci
      shell: bash
```

- Use:

Versioning Actions

- Semantic versioning:
  - `v1`, `v1.1.0`, `v2`
- Recommendations:
  - Tag stable major version: `v1`
  - Update minor/patch with new tags:
    - `v1.1.0`, `v1.1.1`
  - Consumers depend on `v1` for stability

Advanced CI Pipeline Architecture

- Patterns:
  - **Fan-out**:
    - One build job → multiple parallel test jobs
  - **Fan-in**:
    - Multiple jobs → one final report or deploy job
- Use `needs:` to control dependencies

Conditional Job Execution

- Use `if:` for conditions:

```yaml
jobs:
  deploy:
    if: ${{ github.ref == 'refs/heads/main' && github.event_name == 'push' }}
    runs-on: ubuntu-latest
```

- Scenarios:
  - Only deploy from main
  - Only run heavy tests on schedule

Handling Monorepos with Multiple Pipelines

- Strategies:
  - Path filters to trigger specific workflows:

```yaml
on:
  push:
    paths:
      - "backend-node/**"
```

  - Separate workflows for:
    - Backend
    - Frontend
    - Infrastructure
- Prevents:
  - Unnecessary builds when unrelated files change

Advanced GitHub Actions Security

- Principles:
  - **Least privilege** for `GITHUB_TOKEN`
  - Defend against malicious PRs from forks
  - Protect secrets and artifacts

Hardening Permissions

- Restrict `GITHUB_TOKEN`:

```yaml
permissions:
  contents: read
  pull-requests: write
```

- Avoid full `write-all` permissions
- Only grant what the workflow needs:
  - For example, only `packages: write` when pushing images

Handling Untrusted Pull Requests

- PRs from forks:
  - Run workflows with reduced privileges
  - Do not expose deployment secrets
- Techniques:
  - Use separate workflows for:
    - `pull_request` (no secrets)
    - `push` to protected branches (secrets allowed)
  - Require manual approval for dangerous workflows

OIDC Authentication for Cloud Deployment (Preview)

- Instead of static cloud credentials:
  - Use **OpenID Connect (OIDC)**:
    - GitHub Issues a short-lived token
    - Cloud provider (AWS/GCP/Azure) trusts GitHub as identity provider
- Benefits:
  - No long-lived keys in secrets store
  - Stronger security posture

Monitoring, Debugging & Optimization

- Observability for workflows:
  - Understand where time is spent
  - Identify flaky steps
  - Improve developer experience

Interpreting Logs & Matrix Logs

- Per-job logs:
  - Scroll and search for `Error` or `Failed`
- Matrix logs:
  - Each combination (OS, version) has its own log
  - Identify patterns:
    - Fails only on Windows
    - Fails only on Node.js 20
- Use this data to:
  - Fix cross-platform issues

Rerunning Failed Jobs and Debug Mode

- Rerun options:
  - `Re-run all jobs`
  - `Re-run failed jobs`
- Debug logs:
  - Set secret `ACTIONS_STEP_DEBUG` to `true`
  - GitHub prints extra debug information
- Useful for:
  - Intermittent and complex failures

Speed Optimization Checklist

- Use caching effectively
- Avoid redundant work:
  - Build once, reuse artifacts
- Disable unnecessary matrix combinations
- Run heavy tasks on schedule instead of every PR
- Periodically review workflow runtime

Handling Flaky Integration Tests

- Symptoms:
  - Random failures in CI
- Strategies:
  - Add retries at test level (not in CI loop)
  - Improve isolation:
    - Use dedicated test databases
  - Use timeouts carefully
- Goal:
  - CI pipeline should be **predictable and stable**

Summary & Looking Ahead

- You have seen GitHub Actions from:
  - Basic workflows to pro-level architectures
- Key skills:
  - Workflow design, caching, Docker builds
  - Security, reusable workflows, self-hosted runners
- Next steps:
  - Apply these patterns in your course projects
  - Integrate GitHub Actions with CD pipelines and cloud deployments